

Grundlagen – Architektur und Konfiguration von Kubernetes

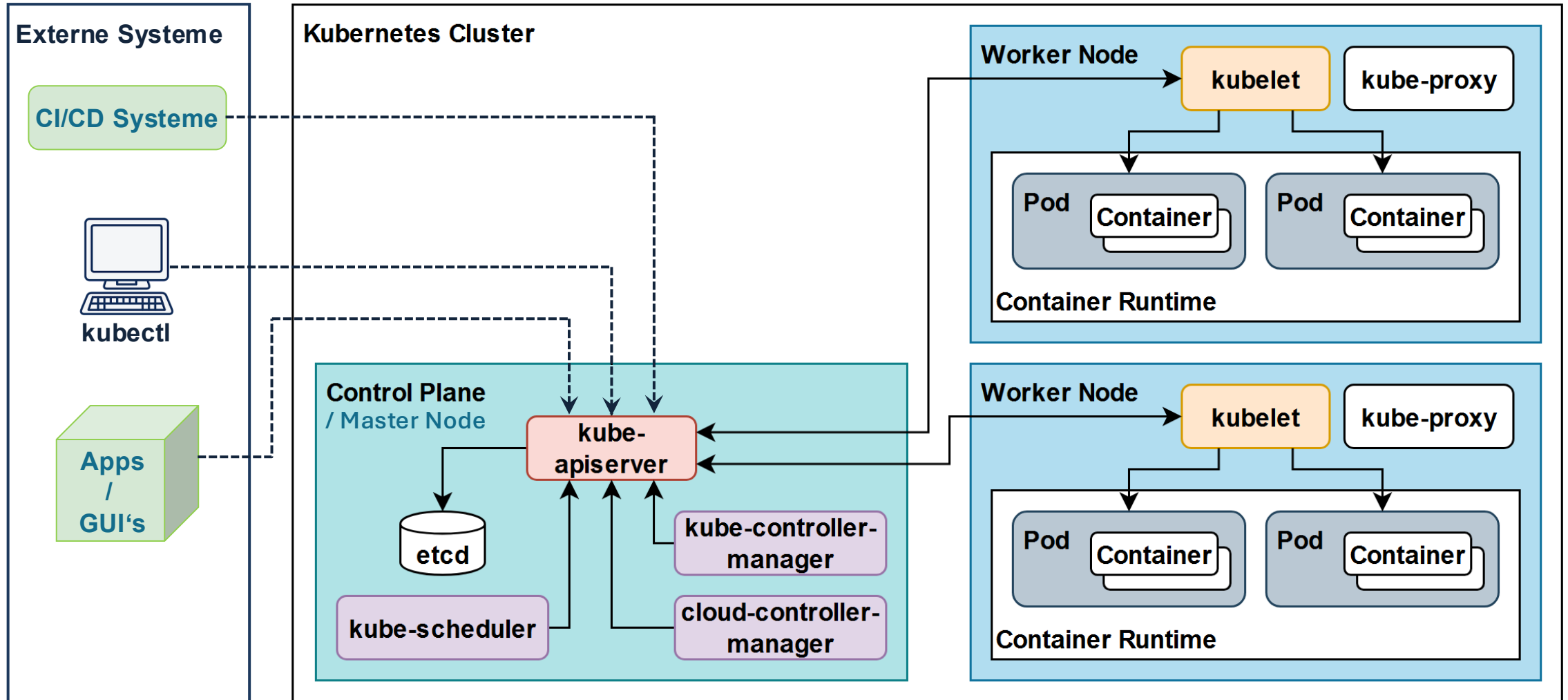
Container mit Schwerpunkt Kubernetes



- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
- ConfigMap und Secrets

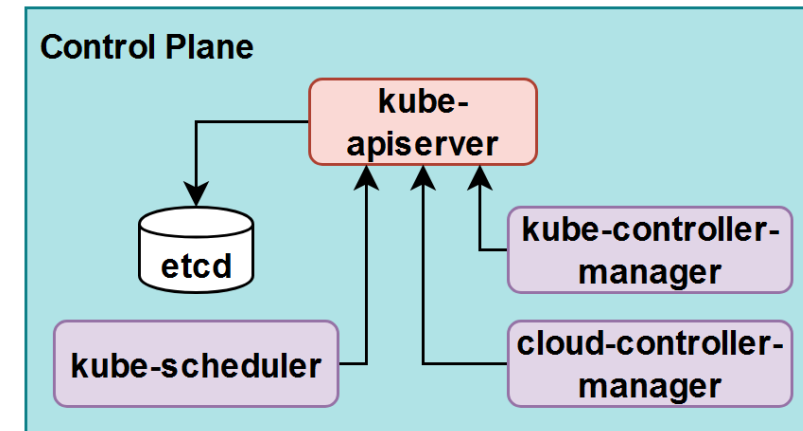
- Grundlegend besteht Kubernetes aus:
 - 1-n Master-Nodes (Control Plane),
 - 1-n Worker-Nodes,
 - einem verteilten Key-Value-Store (etcd).

Übersicht



Control Plane (Master-Node)

- Verantwortlich für das **Management des Kubernetes-Clusters**.
- Eingangspunkt für alle administrativen Tasks.
- Zugriff erfolgt über eine CLI, GUI oder APIs.
- Um die Ausfallsicherheit zu erhöhen, können **mehrere Master-Nodes** in einem Cluster existieren.
- Control Plane-Prozesse werden dann im High-Availability-Modus betrieben.
- **Einer** dieser Prozesse ist der **Leader** und führt alle entsprechenden Operationen aus.



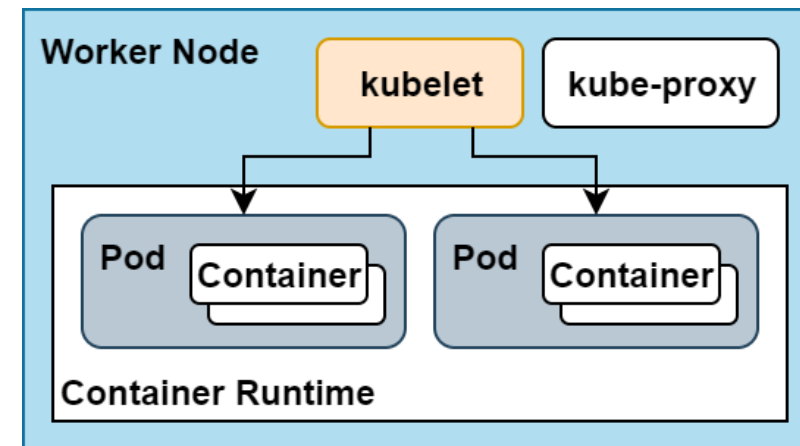
- API-Server (*kube-apiserver*)
 - Der API-Server bietet eine **REST-API**.
 - Der User sendet ein Kommando über die API an den Server, dieser validiert den Request und führt ihn aus.
 - Das **Ergebnis** wird anschließend **im Key-Value-Store hinterlegt**.
 - **Alle administrativen Tasks** werden über den API-Server ausgeführt.
- Scheduler (*kube-scheduler*)
 - **Verteilt** neue **Pods** auf die verschiedenen Worker-Nodes.
 - Dabei werden verschiedene Eigenschaften (**u. a. Ressourcenanforderungen, Constraints und Affinities**) berücksichtigt.

- Controller Manager (*kube-controller-manager*)
 - **Führt Controller aus**, die den Status des Clusters über den API-Server überwachen.
 - Jeder Controller ist für ein **bestimmtes Objekt** (Node, Endpoint, ...) **zuständig**.
 - Weicht der Status eines Objektes vom definierten Status ab, werden **Korrekturmaßnahmen** eingeleitet.
- Cloud Manager (*cloud-controller-manager*)
 - Führt **Controller** aus, die mit der Plattform des **Cloud-Providers** interagieren können.
 - Trennung vom *kube-controller-manager* ermöglicht die **unabhängige Entwicklung** durch die Cloud-Provider.

- Key-Value-Store (*etcd*)
 - **Verteilte Key-Value-Datenbank** für alle Cluster-Daten.
 - Speichert zusätzlich Konfigurationsdetails wie z. B. Subnetze und Secrets und Laufzeitdaten des Clusters.
 - Einer der Nodes im Verbund ist Leader und alle anderen sind Follower.
 - Liefert eigene HTTP JSON API.
 - Unterstützt Verschlüsselung und Authentifizierung.

Worker-Node

- Ein **Worker-Node** wird auch einfach als **Node** bezeichnet.
- Wird **vom Master-Node gesteuert**.
- **Führt** Applikationen unter Verwendung von **Pods aus**.
- Pods werden auf den Nodes eingeplant, die über die notwendigen Eigenschaften für den Betrieb verfügen.
- Ein Pod besteht aus einem oder mehreren Containern.



Worker-Node – Komponenten

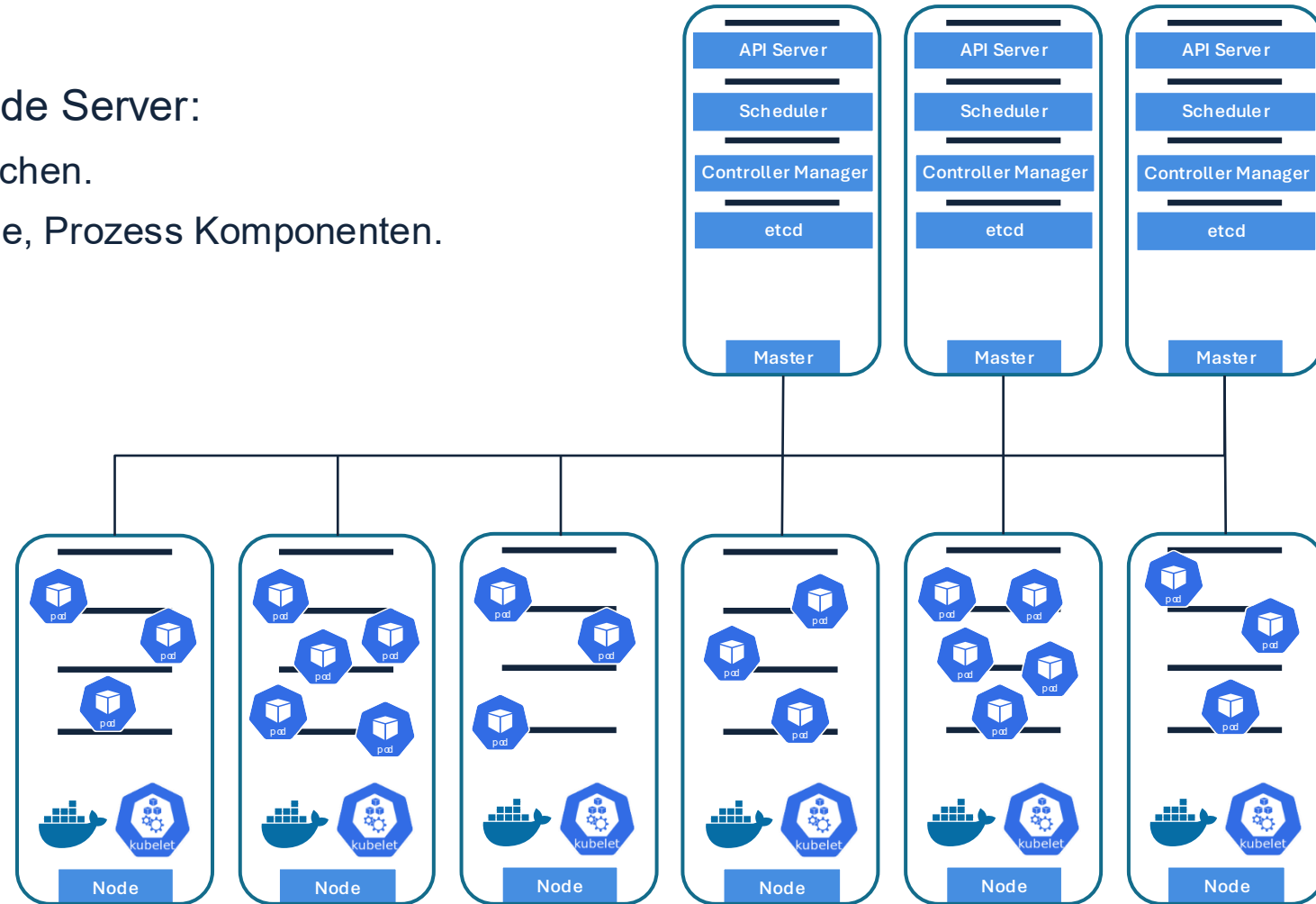
- *kubelet*
 - Ein Agent, der **auf jedem Worker-Node** läuft.
 - Kommuniziert mit dem *kube-api-server*.
 - Stellt sicher, dass die Container in einem Pod laufen und einen definierten Zustand haben.
 - **Kommuniziert mit der Container Runtime** über das Container Runtime Interface (CRI).
- Container Runtime
 - Software, die für das **Ausführen von Containern** verantwortlich ist.
 - Es kann jede Runtime genutzt werden, die das CRI implementiert:
 - z. B. Docker (containerd), CRI-O und rkt.
- *kube-proxy*
 - Ist für das **Verteilen und Weiterleiten von Anfragen über das Netzwerk an die Pods** zuständig.

- Addons erweitern die Funktionalität von Kubernetes.
- Sie sind selber **Pods und Services, die im Cluster laufen.**
- Einige Addons sind quasi Standard.

- Beispiele:
 - DNS,
 - Dashboard (WEB UI),
 - Cluster Ressourcen Monitoring,
 - Cluster Level Logging.

Beispiel Cluster Setup

- Hinzufügen neuer Master/Node Server:
 - 1) Neuen Server verfügbar machen.
 - 2) Installation aller Master/Node, Prozess Komponenten.
 - 3) Zum Cluster hinzufügen.



Kubernetes-Cluster: Arbeitsteilung

Plattform bereitstellen und Anwendungen betreiben

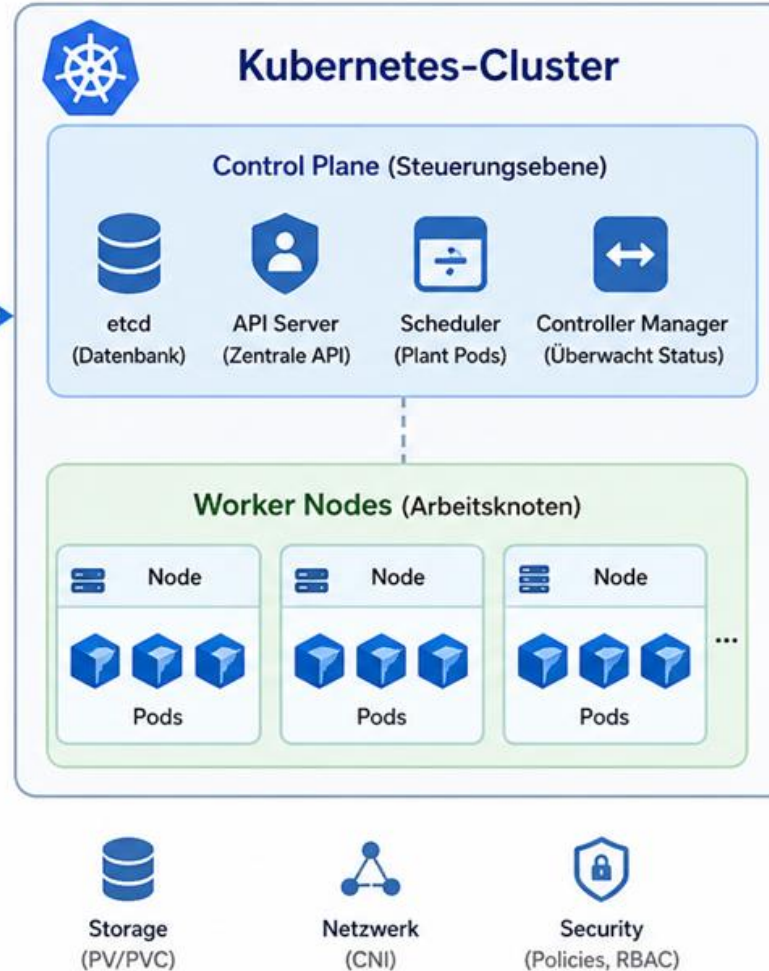
Cluster-Administrator

Stellt den Kubernetes-Cluster bereit und betreibt die Plattform



Verantwortlichkeiten

- Cluster installieren und konfigurieren
- Worker Nodes verwalten
- Netzwerk und Ingress bereitstellen
- Storage bereitstellen und verwalten
- Sicherheit und RBAC konfigurieren
- Updates, Monitoring und Backups



DevOps-Engineer

Nutzt den Cluster für Anwendungen und Services



Verantwortlichkeiten

- Anwendungen containerisieren
- Deployments und StatefulSets erstellen
- Services definieren (ClusterIP, NodePort)
- Ingress und Routing konfigurieren
- Konfigurationen und Secrets verwalten
- Logs, Monitoring und Alerts nutzen



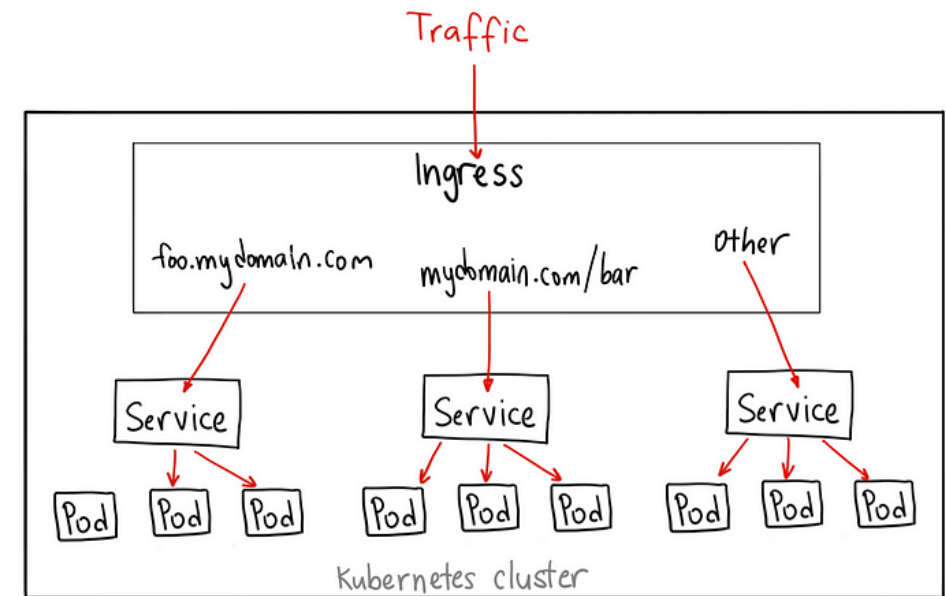
Gemeinsames Ziel: Stabile, sichere und skalierbare Anwendungen auf Kubernetes bereitstellen

Agenda

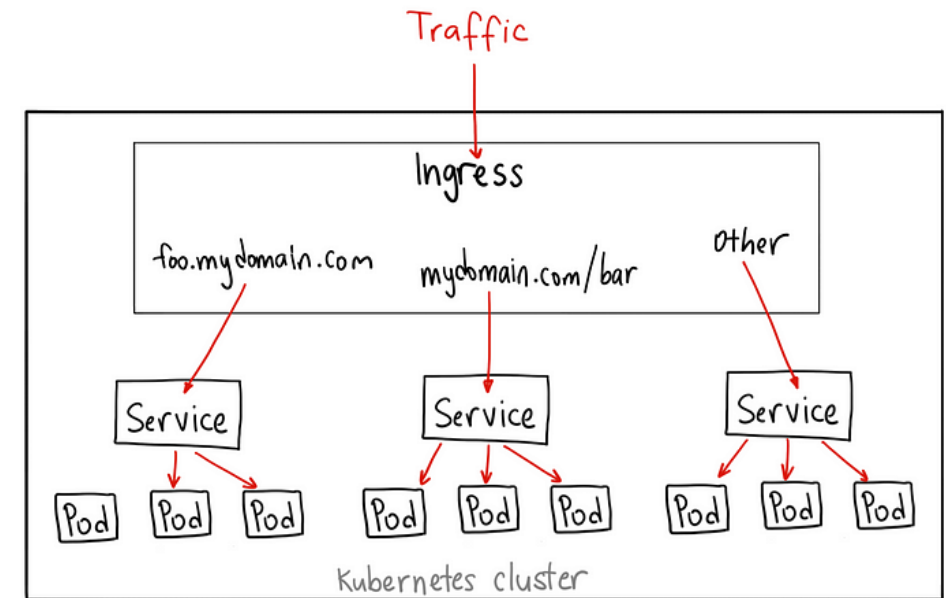
- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
- ConfigMap und Secrets

Interaktion mit Kubernetes für folgende Anwendungsfälle:

- Deployment (Installation) von Anwendungen.
- Management der installierten Anwendungen.
- Beispiele:
 - Deployment: Aufbau eines Anwendungs-Clusters.
 - Service: Interne Erreichbarkeit des Anwendungs-Clusters.
 - Ingress: Externe Erreichbarkeit des Anwendungs-Clusters.
 - Zustandsüberwachung.
 - Fehlerbehebung.



- Kubernetes ordnet Attribute (Konfigurations-)Objekten zu.
- Beispiele:
 - **Deployment:**
 1. Welches Container-Image?
 2. Wie viele Instanzen?
 - **Service:**
 1. Für welche Anwendung?
 2. Welcher Port?
 - **Ingress:**
 1. Für welchen Service?
 2. Nach welcher Regel?



- Ein Manifest ist die Konfigurations-Datei zu einem Objekt.
- Als Format wird in der Regel YAML verwendet
- JSON ist auch möglich.
- Konfiguration durchführen:
kubectl apply -f <Manifest>

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - name: test-pod
    image: busybox
    command:
    - sh
    - -c
    - "echo Hello World! && sleep 180"
```

Interaktion mit Kubernetes - kubectl

- Kubernetes stellt eine Kommandozeilen-Applikation zur Verfügung: **kubectl**.
- kubectl kommuniziert mit dem kube-api-server.
- Die Installation von kubectl ist abhängig vom Betriebssystem:
 - Linux:

```
# curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl  
# chmod +x kubectl  
# sudo mv kubectl /usr/local/bin/
```
 - Auto-Vervollständigung mit bash: `source <(kubectl completion bash)`
 - In .profile konfigurieren: `echo "source <(kubectl completion bash)" >> ~/.profile`

kubectl – Kubeconfig (1)

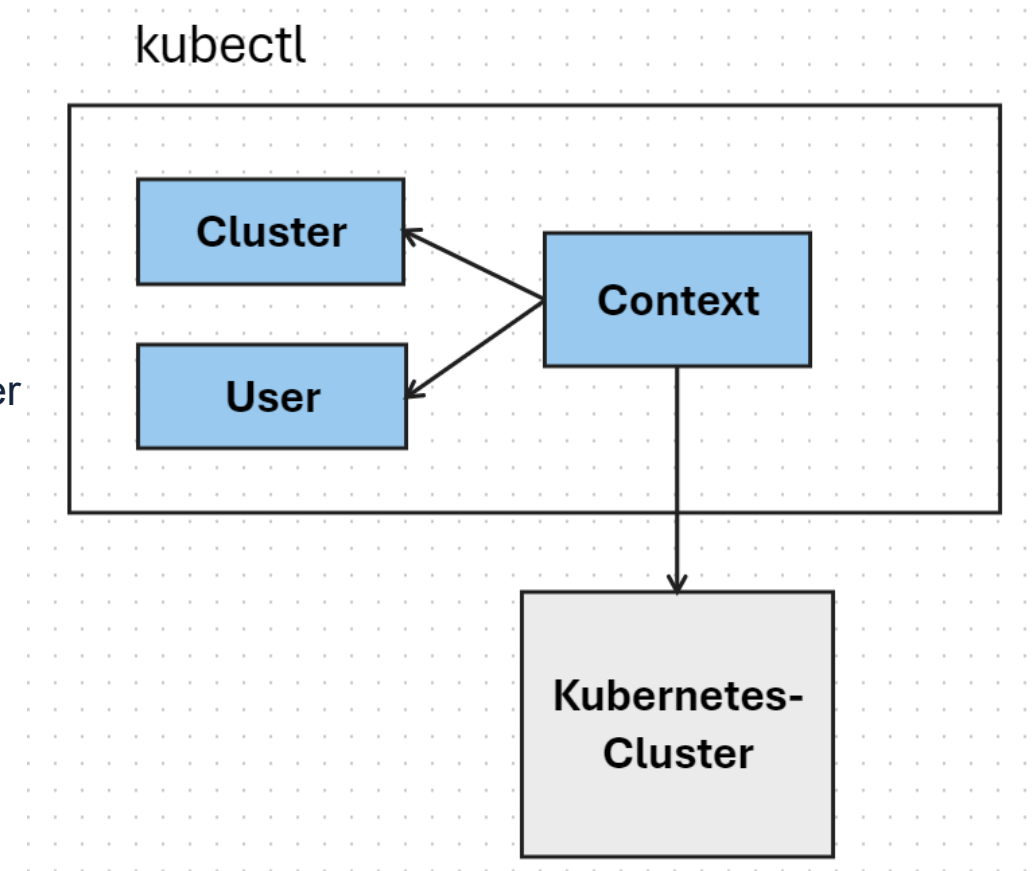
- Die Konfigurationsdatei für kubectl (Kubeconfig) ist **\$HOME/.kube/config**.
- Alternative Konfigurations-Dateien können ausgewählt werden:
 1. mit der Umgebungsvariablen **KUBECONFIG**.
 2. durch Angabe des Parameters **--kubeconfig <Konfigurations-Datei>**.
- Die Syntax von Kubeconfig überprüfen Sie mit dem Befehl:
`# kubectl config view`
- Mit dem folgenden Kommando können Sie die Verbindung zum Kubernetes-Cluster überprüfen:
`# kubectl cluster-info`

Im Fehlerfall erscheint eine Meldung wie:

The connection to the server <server-name:port> was refused - did you specify the right host or port?

kubectl – Kubeconfig (2)

- Die Bestandteile einer Kubeconfig-Datei:
 - 1. Cluster-Konfiguration:**
`# kubectl config get-clusters`
Enthält Verbindungsinformationen zu Kubernetes-Clustern.
 - 2. Benutzer-Konfiguration:**
`# kubectl config get-users`
Definiert die verschiedenen Benutzer, die sich mit dem Cluster authentifizieren können.
 - 3. Kontext-Konfiguration:**
`# kubectl config get-contexts`
Kombiniert Cluster- und Userinformationen.
 - 4. Aktueller Kontext:**
`# kubectl config current-context`
Definiert den Standardkontext.



- Kubeconfig-Datei kann für mehrere Kubernetes-Cluster konfiguriert werden:

1. Hinzufügen von Clusterinformationen:

```
# kubectl config --kubeconfig=config-demo set-cluster development  
--server=https://1.2.3.4  
--certificate-authority=fake-ca-file
```

2. Hinzufügen eines neuen Users:

```
# kubectl config --kubeconfig=config-demo set-credentials developer  
--client-certificate=fake-cert-file  
--client-key=fake-key-seefile
```

3. Hinzufügen eines neuen Contexts:

```
# kubectl config --kubeconfig=config-demo set-context dev-frontend  
--cluster=development -namespace=frontend  
--user=developer
```

- Ermöglichen **Unterteilung eines Kubernetes-Clusters** in Sub-Cluster.
- **Namen für Objekte sind in einem Namensraum eindeutig!**
- Kubernetes verfügt über 4 vordefinierte Namespaces:
 - *default* Default Namespace für Objekte, die zu keinem anderen Namespace gehören.
 - *kube-system* Namespace für Objekte, die vom Kubernetes-System erstellt wurden.
 - *kube-public* - Für jeden lesbarer Namespace.
 - Für die interne Nutzung reserviert.
 - Wird benötigt, um ein Cluster zu erstellen.
 - *kube-node-lease* Wird für Heartbeats zwischen den Nodes genutzt.
- Einige Ressourcen gehören immer zu einem Namespace (z. B. *Pod*, *ReplicaSet*, *Secret*, *Deployment*), andere leben außerhalb von Namespaces (z. B. *Node*, *Volume*).

kubectl – Arbeiten mit Namespaces

- **Abfragen vorhandener Namespaces:**
`# kubectl get namespaces | ns`
- **Namespace bei einer Anfrage angeben:**
`# kubectl --namespace=kube-system get pods`
- **Erstellen eines neuen Namespace:**
`# kubectl create ns <name>`
- **Wechseln in einen vorhandenen Namespace:**
`# kubectl config set-context --current --namespace=<name>`
- **Auflisten der an einen Namespace gebundenen Ressourcen:**
`# kubectl api-resources --namespaced=true`

Namespaces helfen, Ressourcen innerhalb eines Clusters zu organisieren:

- **Trennung von Umgebungen (Unterschiedliche Umgebungen):**

1. Verwenden Sie verschiedene Namespaces für unterschiedliche Umgebungen wie *dev*, *staging* und *production*.

```
kubectl create namespace dev  
kubectl create namespace staging  
kubectl create namespace production
```

- **Ressourcenquoten und Limits (Ressourcenquoten setzen):**

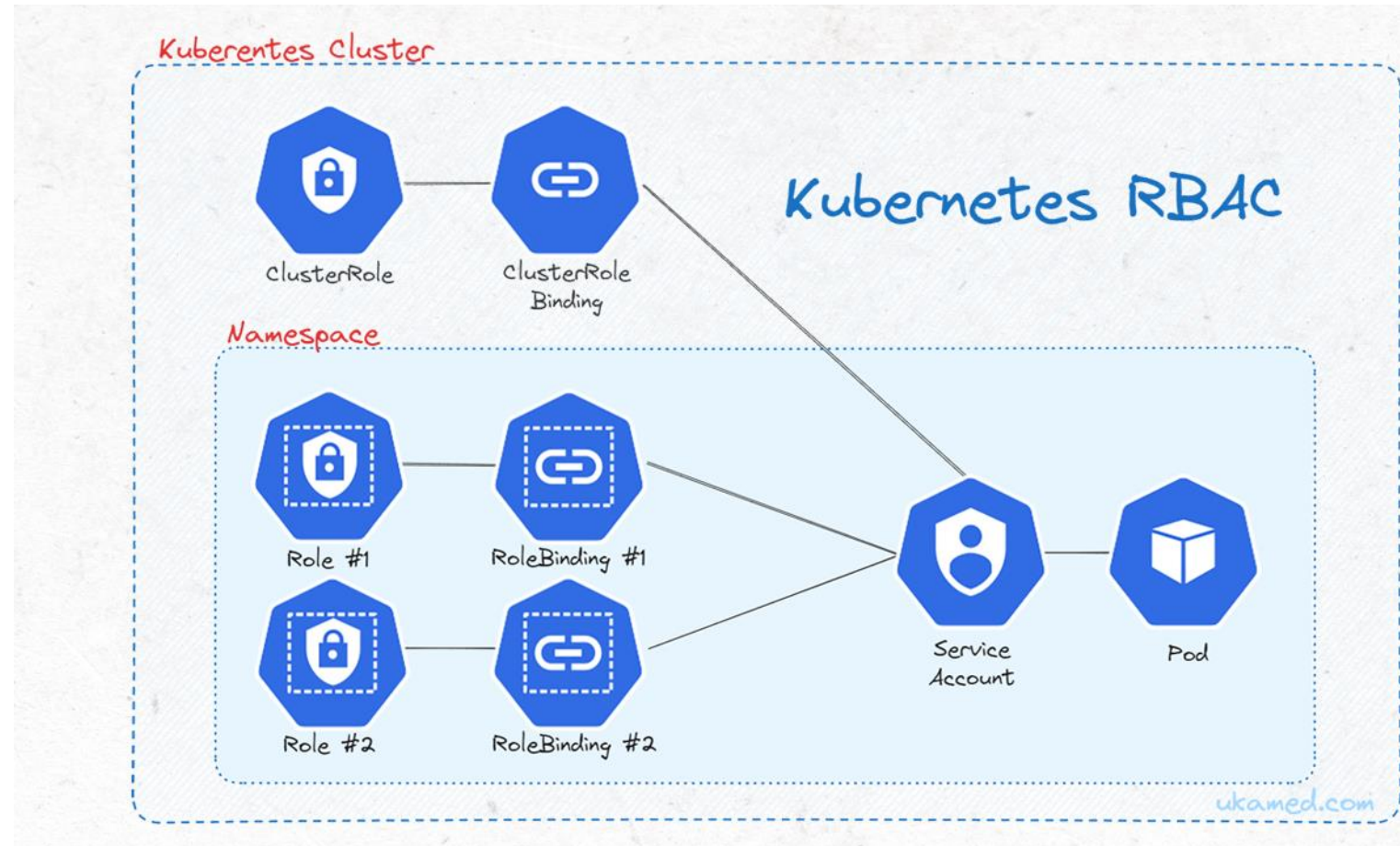
1. Wenden Sie Ressourcenquoten auf Namespaces an, um die Menge an Ressourcen (CPU, Speicher usw.), die verbraucht werden können, zu kontrollieren.

```
apiVersion: v1  
kind: ResourceQuota  
metadata:  
  name: resource-quota  
  namespace: dev  
spec:  
  hard:  
    requests.cpu: "1"  
    requests.memory: 1Gi  
    limits.cpu: "2"  
    limits.memory: 2Gi
```

- **RBAC (Role-Based Access Control) Zugriff kontrollieren:**
 1. Verwenden Sie RBAC, um zu definieren, welche Benutzer und Service-Accounts innerhalb eines Namespaces was tun dürfen.
 2. Definieren Sie Rollen und Rollenbindungen für namespacespezifische Berechtigungen.

Berechtigung in Kubernetes

- **RBAC (Role-Based Access Control)** wird in Kubernetes durch Rollen und Role-Bindings umgesetzt.
- **Rollen** (Roles) fassen Berechtigungen zusammen.
- **Role-Bindings** vergeben Rollen an einen Benutzer oder einen Service-Account.



- **Namespace erstellen:**
`$ kubectl create namespace abc`
- **Namespaces auflisten:**
`$ kubectl get namespaces`
`$ kubectl apply -f <Manifest>`
- **Namespaces löschen:**
`$ kubectl delete namespace <Name>`
- **Default-Namespace setzen:**
`$ kubectl config set-context --current --namespace=my-namespace`
- **Rollen fassen Berechtigungen zusammen.**
- **Rollen auflisten:** `$ kubectl get role`
- **Beschreibung:**
`$ kubectl describe role pod-reader`
- **Löschen:** `$ kubectl delete role pod-reader`

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace

---

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: my-namespace
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

- Berechtigungen beziehen sich auf Objekte.
- **apiGroup:** Jedes Objekt gehört zu einer API-Group - erster Teil der apiVersion.
kubectl api-resources
- **resources:** Bezeichnet die Objekte.
- **verbs:** Aktionen auf das Objekt:
 - **get:** Allows reading the details of a resource.
 - **list:** Allows listing all resources of a given type
 - **watch:** Allows watching for changes to resources.
 - **create:** Allows creating a new resource.
 - **update:** Allows modifying an existing resource.
 - **patch:** Allows partially modifying a resource using patches.
 - **delete:** Allows deleting a resource.
 - **edit:** Grants permission to update and patch resources

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: my-namespace
  name: pod-reader
rules:
- apiGroups: [""]          # Core API-Gruppe
  resources: ["pods"]
  verbs: ["get", "list"]
```

Erstellen eines Service-Accounts mit dem Namen *my-service-account*:

- **apiVersion** gibt Manifest-Version an.
- **kind** definiert Typ als ServiceAccount.
- **metadata** enthält Metadaten wie Name und Namespace

Erstellen: # `kubectl apply -f <Manifest>`

Auflisten: # `kubectl get serviceaccounts`

Beschreibung:

```
# kubectl describe serviceaccounts  
my-service-account
```

Löschen:

```
# kubectl delete serviceaccounts  
my-service-account
```

```
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: my-service-account  
  namespace: my-namespace
```

- **RoleBinding** vergibt Berechtigungen einer Rolle (*roleRef*) an einen oder mehrere Benutzer (*subjects*), z. B. einen Service-Account.
 - **Erstellen:** `# kubectl apply -f <Manifest>`
 - **Auflisten:** `# kubectl get rolebindings.rbac.authorization.k8s.io`
 - **Beschreibung:**
`# kubectl describe rolebindings.rbac.authorization.k8s.io pod-reader-binding`
 - **Löschen:**
`# kubectl delete rolebindings.rbac.authorization.k8s.io pod-reader-binding`

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: my-namespace
subjects:
  - kind: ServiceAccount
    name: my-service-account
    namespace: my-namespace
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

- Anstelle eines Role-Bindings wird ein **ClusterRoleBinding** verwendet:
Das Cluster Role-Binding gilt unabhängig vom Namespace.
- Anstelle eines Service-Accounts wird ein User verwendet.
 - Erstellen: # `kubectl apply -f <Manifest>`
 - Auflisten: # `kubectl get clusterrole`
 - Beschreibung:
`kubectl describe clusterrole <Name>`

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-cluster-role-binding
subjects:
  - kind: User
    name: alice
roleRef:
  kind: ClusterRole
  name: my-cluster-role
  apiGroup: rbac.authorization.k8s.io
```

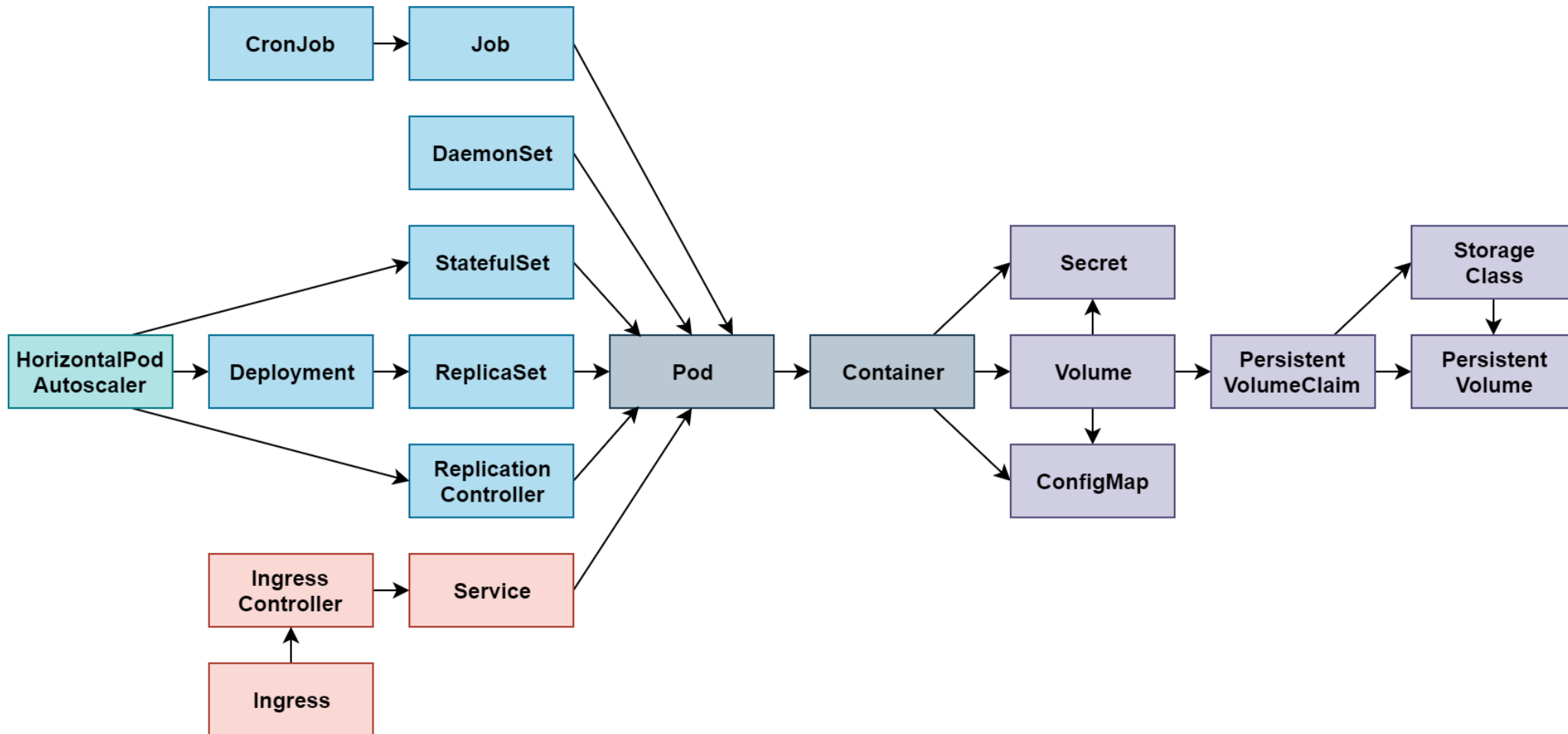
- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
 - Umgang mit Pods
 - Pods überwachen - Probes
 - Ressourcen-Limitierung
- ConfigMap und Secrets

Agenda

- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
 - Umgang mit Pods
 - Pods überwachen - Probes
 - Ressourcen-Limitierung
- ConfigMap und Secrets

- Kleinstes deploybares Kubernetes-Objekt.
- **Logische Zusammenfassung einer oder mehrerer Container.**
- Werden gewöhnlich von einem Deployment verwaltet.
- Bilden **eine Instanz einer Applikation.**
- Container in einem Pod teilen dieselben Netzwerk-Namespaces und Volumes, ihre Ressourcen werden jedoch durch Control Groups getrennt.
 - werden auf demselben Node gestartet
- *Pods* sind **nicht beständig** und haben nicht die Fähigkeit, sich selbst zu heilen.
- Controller gewährleisten die Replikation, Fehlertoleranz und Selbstheilung.

Objekte – Pods im Verbund mit den weiteren Objekten



Objekte – Spezifikationen und Status

- Kubernetes-Objekte beinhalten **zwei Objekt-Felder**
- Objektspezifikation
 - Wird **vom Benutzer angegeben**.
 - **Beschreibt den Wunschzustand** (Eigenschaften), die das Objekt haben soll.
- Objektstatus
 - Beschreibt den **Ist-Zustand** des Objektes.
 - Wird vom Kubernetes-System geliefert und aktualisiert.
 - Kubernetes stellt sicher, dass der Ist-Zustand mit dem Soll-Zustand übereinstimmt.

Objekte – Beschreibung (Manifest)

- **Objektspezifikation muss einige Basisinformationen enthalten.**
- `kubectl` nutzt YAML- oder JSON-Dateien zur Beschreibung von Objekten.
- `kubectl` wandelt diese - wenn notwendig - in JSON um und sendet sie an die API.
- **Pflichtfelder:**

<i>apiVersion</i>	Verwendete Kubernetes-API Version.
<i>kind</i>	Typ des Objektes.
<i>metadata</i>	Informationen (z. B. Name) zur Objektidentifikation.
<i>spec</i>	Spezifikation des Objektes (hängt von dessen Art ab).
- **Siehe auch:** `kubectl explain <type>[.<fieldname>[...]]`
Beispiel: `kubectl explain pod.status`

Objekte – Manifest – YAML-Beispiel

- Erstellen einer Ressource (Pod) aus einem Manifest:

```
$ kubectl create|apply -f pod.yml
$ kubectl logs test-pod
Hello World!
```

- Starten eines Pods unter Angabe eines Images:

```
$ kubectl run test-pod --restart=Never
--image=busybox \
  -n userXX --command -- sh -c "echo
Hello World!"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - name: test-pod
    image: busybox
    command:
    - sh
    - -c
    - "echo Hello World! && sleep 180"
```

- Auflisten der Pods:

```
# kubectl get pod | po
```

NAME	READY	STATUS	RESTARTS	AGE
test-pod	1/1	Running	0	33s
...				

- Details zu einem Pod anzeigen:

```
# kubectl describe pod test-pod
```

Name: test-pod
Namespace: default
Node: minikube/192.168.99.100
...

- Objektspezifikation und -status ausgeben:

```
# kubectl get pod test-pod -o yaml
```

```
apiVersion: v1
...
```

- Löschen eines Pods:

```
# kubectl delete pod test-pod
```

```
pod "test-pod" deleted
```

```
# kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
test-pod	1/1	Terminating	0	33s

kubectl – Zugriff auf den Pod

- stdout und stderr der Applikationen im Container anzeigen:

```
# kubectl logs test-pod  
# kubectl logs [-f] [-p] <pod> [-c <container>]
```

- Kommando in einem Container ausführen:

```
# kubectl exec test-pod -- date  
# kubectl exec <pod> [-c <container>] [-i] [-t] -- <command>
```

- Kopieren von Dateien und Verzeichnissen in einen Container und vice versa:

```
# kubectl cp test-pod:/etc/passwd ./passwd  
# kubectl cp <src> <dest> [-c <container>]
```

- Einfache Port-Weiterleitung:

```
# kubectl run test2-pod --restart=Never --image=nginx  
# kubectl port-forward test2-pod 1234:80  
# kubectl port-forward <pod> [--address <ip>] \ <local-port>:<remote-port>
```

- Erzeugt wird ein Pod mit dem Container-Image hashicorp (image: hashicorp/http-echo:latest).
- Der zurückgegebene Text soll 1.0.0 sein: (containers.args[0] = -text=1.0.0)
- Hashicorp hat standardmäßig einen Listener auf den Port 5678.

- **Struktur des Manifests erzeugen mit:**

```
$ kubectl run hashicorp-http-echo \
  --image=hashicorp/http-echo:latest \
  --dry-run=client \
  -o yaml > hashicorp-1.1.0-pod.yaml
```

- Manifest anpassen (s. rechts).

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: hashicorp-http-echo
  name: hashicorp-http-echo
spec:
  containers:
    - image: hashicorp/http-echo:latest
      name: hashicorp-http-echo
      args:
        - "-text=1.0.0"
      ports:
        - containerPort: 5678
```

- Der Service kann mit *kubectl port-forward* direkt von der Shell aus angesprochen werden.
- Diese Funktionalität wird nur zu Test-Zwecken verwendet.

```
$ kubectl -n user-sk port-forward  
hashicorp-http-echo 8081:5678 &  
[1] 2106  
Forwarding from 127.0.0.1:8081 -> 5678  
Forwarding from [::1]:8081 -> 5678  
  
$ curl http://localhost:8081  
Handling connection for 8081  
1.0.0
```

- Der Pod ist im Cluster über seine IP-Adresse erreichbar.

```
$ kubectl -n user-sk get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	...
hashicorp-http-echo	1/1	Running	0	25m	<u>10.44.2.14</u>	...

- Diese IP-Adresse steht im Cluster zur Verfügung.

```
$ kubectl run curl-test --image=curlimages/curl -it --rm -n user-sk -- sh
~ $ curl http://10.44.2.14:5678
1.0.0
```

kubectl – Umgang mit Pods - Beispiel hashicorp - Logs

- Die Logs (Standard-Ausgabe) der Anwendung lassen sich analysieren.

```
$ kubectl -n user-sk logs hashicorp-http-echo
2026/04/13 05:36:43 [INFO] server is listening on :5678
2026/04/13 05:46:57 localhost:8081 127.0.0.1:55688 "GET / HTTP/1.1" 200 6
"curl/7.81.0" 93.863µs
2026/04/13 06:06:07 10.44.2.14:5678 10.44.2.15:35650 "GET / HTTP/1.1" 200 6
"curl/8.19.0" 23.189µs
2026/04/13 06:06:15 10.44.2.14:5678 10.44.2.15:58994 "GET / HTTP/1.1" 200 6
"curl/8.19.0" 16.118µs
```

kubectl – Umgang mit Pods - Beispiel hashicorp - Zustand

- Zustand des Pods .

```
$ kubectl -n user-sk get pod
NAME                                READY   STATUS    RESTARTS   AGE
hashicorp-http-echo                1/1     Running   0           37m
$ kubectl -n user-sk describe pod hashicorp-http-echo
Name:                                hashicorp-http-echo
...
Events:
  Type     Reason          Age   From                      Message
  ----     -
  Normal   Scheduled       38m   default-scheduler        Successfully assigned user-sk/hashicorp-http-echo to gke-kubernetes-clust-kubernetes-clust-04e76e48-7mlz
...
  Normal   Pulling         38m   kubelet                   spec.containers{hashicorp-http-echo}: Pulling image "hashicorp/http-echo:latest"
  Normal   Started         38m   kubelet                   spec.containers{hashicorp-http-echo}: Container started
```

Agenda

- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
 - Umgang mit Pods
 - Pods überwachen - Probes
 - Ressourcen-Limitierung
- ConfigMap und Secrets

Phase	Beschreibung
Pending	Der Pod wurde vom System akzeptiert aber noch nicht erstellt (z. B. Image wird noch heruntergeladen)
Running	Mindestens einer der Container des Pods läuft aktuell oder ist im Zustand starting oder restarting
Succeeded	Alle Container des Pods wurden erfolgreich beendet und werden nicht neu gestartet
Failed	Alle Container des Pods wurden beendet, mindestens einer jedoch fehlerhaft
Unknown	Der Status des Pods konnte, z. B. aufgrund von Kommunikationsproblemen mit dem Node, nicht ermittelt werden

- Der Status eines Pods enthält ein phase-Feld, dass den High-Level-Status des Pods im Lifecycle enthält.
- **Hinweis:** Status der `kubectl get` Ausgabe entspricht nicht der Phase.

```
# kubectl get po test-pod --no-headers -o custom-columns=":status.phase"
```

- Health Checks monitoren den Zustand von Containern.
- Kubernetes nutzt das Ergebnis, um die Quality of Service zu optimieren.
- Das BSI empfiehlt ausdrücklich in der APP.4.4.A11 die Verwendung von Health Checks.

APP.4.4.A11 Überwachung der Container (S)

In Pods SOLLTE jeder Container einen Health Check für den Start und den Betrieb („readiness“ und „liveness“) definieren. Diese Checks SOLLTEN Auskunft über die Verfügbarkeit der im Pod ausgeführten Software geben. Die Checks SOLLTEN fehlschlagen, wenn die überwachte Software ihre Aufgaben nicht ordnungsgemäß wahrnehmen kann. Für jede dieser Kontrollen SOLLTE eine dem im Pod betriebenen Dienst angemessene Zeitspanne definieren. Auf Basis dieser Checks SOLLTE Kubernetes die Pods löschen oder neu starten.

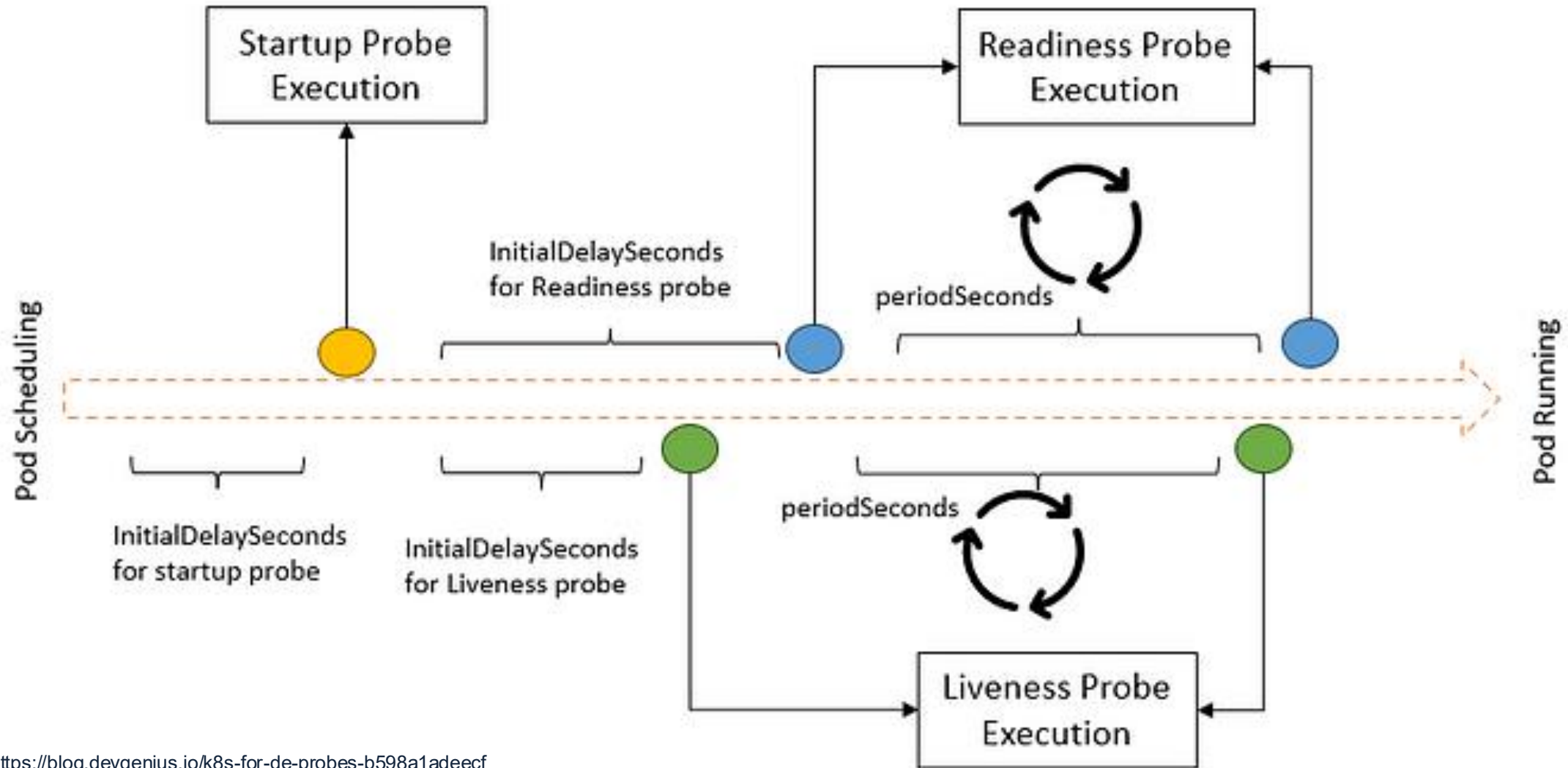
Quelle:

„[https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Grundschatz/IT-GS-](https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Grundschatz/IT-GS-Kompendium_Einzel_PDFs_2022/06_APP_Anwendungen/APP_4_4_Kubernetes_Edition_2022.pdf?__blob=publicationFile&v=3)

[Kompendium_Einzel_PDFs_2022/06_APP_Anwendungen/APP_4_4_Kubernetes_Edition_2022.pdf?__blob=publicationFile&v=3](https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Grundschatz/IT-GS-Kompendium_Einzel_PDFs_2022/06_APP_Anwendungen/APP_4_4_Kubernetes_Edition_2022.pdf?__blob=publicationFile&v=3)“

- Probe ist ein **periodischer Test**, der vom kubelet **im Container ausgeführt** wird.
- Dafür ruft kubelet einen Handler auf, der vom Container implementiert wird.
- Mögliche Handler:
 - *exec* Kommando im Container ausführen.
 - *tcpSocket* TCP-Check gegen definierte IP/Port durchführen.
 - *httpGet* HTTP-GET-Request gegen definierte IP/Port und Path durchführen.
- Mögliche Probes:
 - *startupProbe* Ist die Applikation im Container gestartet?
 - *livenessProbe* Ist ein Container im Status „Running“?
 - *readinessProbe* Kann ein Container Requests beantworten?

Pod – Readiness-Probe / Startup-Probe



<https://blog.devgenius.io/k8s-for-de-probes-b598a1adeecf>

readinessProbe

- Ist der Container in der Lage, Anfragen zu beantworten?
- Schlägt die Überprüfung fehl, wird die IP des Pods von den Endpoints aller Services, die den Pod bedienen, entfernt.
- Ermöglicht dem Container z. B. sich selbst in Maintenance zu nehmen.
- Wird regelmäßig in einem definierten Intervall ausgeführt.

startupProbe

- Benötigt der Container länger als *initialDelaySeconds* + *failureThreshold* x *periodSeconds* zum Starten, sollte ein *startupProbe* definiert werden.
- Nutzt den gleichen Handler wie die *livenessProbe*.
- Alle anderen Probes werden disabled, bis der *startupProbe* erfolgreich ist.
- Schlägt die Überprüfung fehl, wird der Container getötet und entsprechend der *restartPolicy* behandelt.
- Wird nur während des Starts eines Pods (Containers) ausgeführt.

Pod – Health Checks – livenessProbe - Beispiel exec

- Beispiel: Die Anwendung ist live, solange es die Datei `/tmp/hlth` gibt.
- Abschnitt *`livenessProbe`* definiert, wie liveness überprüft wird.
- `exec`: Prüfung mit Kommando.
- Rückgabewert des Kommandos entscheidet: 0 ok, sonst not-ok.

Defaults:

- *`restartPolicy`*: *`Always`*
- *`failureThreshold`*: *`3`*
- *`periodSeconds`*: *`10`*

```
apiVersion: v1
kind: Pod
metadata:
  name: mein-pod
spec:
  restartPolicy: OnFailure
  containers:
  - name: stay-healthy-for-30-seconds
    image: busybox
    args:
    - /bin/sh
    - -c
    - touch /tmp/hlth; sleep 10; rm /tmp/hlth; sleep 300
    livenessProbe:
      exec:
        command:
        - cat
        - /tmp/hlth
```

Pod – Health Checks – livenessProbe - Beispiel tcpSocket

- Beispiel: Die Anwendung ist live, solange der Port 80 geöffnet ist.
- Abschnitt *livenessProbe* definiert, wie liveness überprüft wird.
- *tcpSocket*: Existiert der Socket?

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  containers:
  - name: example-container
    image: nginx:latest
    ports:
    - containerPort: 80
    livenessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
```

Pod – Health Checks – livenessProbe - Beispiel httpGet

- Beispiel: Die Anwendung ist live, solange ein httpGet eine Antwort HTTP-200 liefert.
- *httpGet*: Definiert den probe-Request.
- *timeoutSeconds*: Wartezeit auf die Antwort (in Sekunden).
- Detailinformationen verfügbar mit *kubectl get pod <Name> -o yaml*

```
kind: Pod
metadata:
  name: example-pod
spec:
  containers:
  - name: example-container
    image: nginx:latest
    ports:
    - containerPort: 80
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
      timeoutSeconds: 2
```

Probe - weitere Parameter

periodSeconds (default: 10)

- Gibt an, in welchen Intervallen die Liveness-Probe ausgeführt wird.

timeoutSeconds (default: 1)

- Legt fest, wie lange auf eine Antwort des Get-Requests gewartet wird.

failureThreshold (default: 3)

- Konfiguriert die Anzahl der aufeinanderfolgenden Fehlversuche, bevor die Liveness-Probe als fehlgeschlagen bewertet wird.

successThreshold (default: 1)

- Nur relevant für die Readiness-Probe: Konfiguriert die Anzahl der aufeinanderfolgenden Fehlversuche, bevor die Liveness-Probe als fehlgeschlagen bewertet wird.

Agenda

- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
 - Umgang mit Pods
 - Pods überwachen - Probes
 - Ressourcen-Limitierung
- ConfigMap und Secrets

- Ressourcen Management ist für die Quality-of-Service **eines Kubernets-Clusters** essenziell.
- Der Kubernetes-Cluster garantiert den Pods die angeforderten Ressourcen.
- Der Scheduler stellt beim Platzieren eines Pods sicher, dass der Node die Ressourcenanforderungen erfüllt.
- Beim Optimieren der Ressourcen kann Kubernetes einzelne Pods auf einen anderen Workernode umziehen.
- Ist das Memory-Limit überschritten, wird der jeweilige Prozess im Container terminiert und neu gestartet.
- In Kubernetes gibt es folgende QoS-Klassen:
 - **Guaranteed** (Garantiert): Die Anfragen und Limits eines Containers für alle Ressourcen sind identisch. Die angeforderten Ressourcen werden ihm garantiert.
 - **Burstable**: Die Anfragen eines Containers sind geringer als seine Limits. Das bedeutet, dass er mehr Ressourcen nutzen kann, wenn diese verfügbar sind, aber bei hoher Auslastung des Kubernetes-Clusters gedrosselt wird.
 - **BestEffort**: Wenn ein Container keine Anfragen oder Limits gesetzt hat, erhält er die QoS-Klasse "BestEffort". Das macht ihn zum Kandidaten für das Eliminieren bei Ressourcenknappheit.

- Für einen Container können **2 unterschiedliche Ressourcenangaben** definiert werden:
 - *requests* Minimal benötigte Ressourcen
 - *limits* Maximal nutzbare Ressourcen
- Die *requests/limits* für den Pod sind die Summe der Werte für die Container im Pod.
- Mögliche Ressourcentypen:
 - `cpu`
 - `memory`
 - `hugepages-<size>`

Pod – Ressourcen Management (2/2)

- Im Beispiel hat der Pod einen Mindestbedarf von 0,5 CPU und 50 Megabyte. Der maximale Bedarf ist 1 CPU und 100 Megabyte.
- CPU-Bedarf wird in ganzen CPUs (=Kernen) oder aber in Milli-CPU's mit 1000m = 1 CPU angegeben.
- Memory-Angabe dezimal oder binär:
1Ki = 1024 Bytes aber 1K = 1000 Bytes
Dito für Mega- (M) oder Gigabyte (G).
- Die aktuelle Ressourcennutzung anzeigen:
kubectl top po

NAME	CPU(cores)	MEMORY(bytes)
test2-pod	0m	2Mi

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "50Mi"
        cpu: "500m"
      limits:
        memory: "100Mi"
        cpu: 1
```

Pod – Ressourcen Management – Speicher-Überschreitung

- Beispiel: Pod mit Memoy-Leak.
- Programm fordert immer mehr Speicher an.
- Speicher-Limit wird überschritten.
- Kubernetes beendet den Pod:

```
$ kubectl get pod -w
```

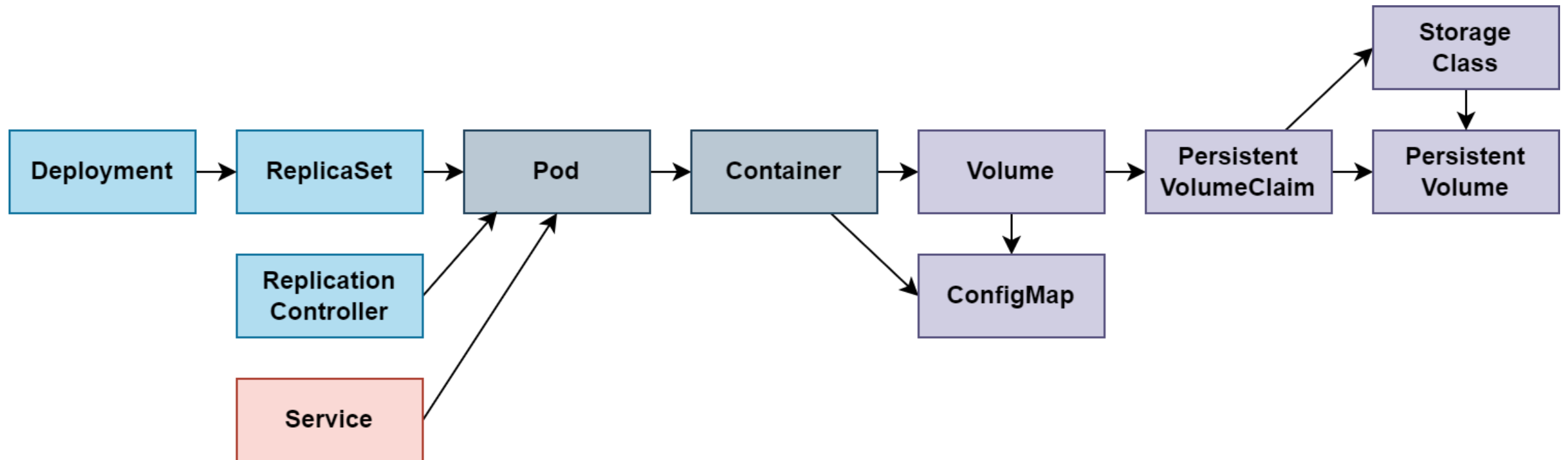
NAME	READY	STATUS	...
memory-limit-test	0/1	ContainerCreating	
memory-limit-test	1/1	Running	
memory-limit-test	0/1	<u>OOMKilled</u>	

```
apiVersion: v1
kind: Pod
metadata:
  name: memory-limit-test
spec:
  containers:
  - name: memory-hog
    image: python:3.9
    command:
    - python3
    - -c
    - |
      a = []
      while True:
        a.append(a * 10**6)
  resources:
    limits:
      memory: "200Mi"
    requests:
      memory: "200Mi"
  restartPolicy: Never
```

Agenda

- Architektur des Kubernetes Cluster
- Konfiguration des Kubernetes Cluster
- Pods
- ConfigMap und Secrets

ConfigMaps - Zusammenhang



- *ConfigMap* ermöglicht die **Trennung der Konfiguration von Container Images**.
- Damit können Images generisch erstellt werden und bleiben portabel.
- Konfigurationsdetails werden als **Key-Value-Paare** definiert.
- Können von Pods und anderen Komponenten (z. B. Controllern) genutzt werden.
- Müssen vor den Ressourcen, die sie verwenden, erstellt werden.
- Sind **an einen Namespace gebunden**.

- **Wichtig:** Daten werden unverschlüsselt abgelegt!

- Möglichkeiten *ConfigMaps* zu erstellen:
 - Key-Value-Paare über die Kommandozeile.
 - Datei: Dateiname (Key), Inhalt (Value).
 - Ordner: Dateinamen (Keys), Inhalte (Values).
 - ENV-Datei: Variablenname (Key) , Variablenwert (Value).
 - YAML-Datei.
- Beispiele:

```
# kubectl create configmap firma-config --from-literal=firma="ORDIX AG"
# kubectl create configmap firma-config --from-file=firma.json
# kubectl create configmap firma-config --from-file=dir/
# kubectl create configmap firma-config --from-env-file=firma.properties
```

```
$ kubectl describe configmaps firma-config
```

- *ConfigMap* imperativ erstellen:

```
$ kubectl create configmap firma-config \
--from-literal=firma="ORDIX AG" \
--from-literal=strasse="Karl-Schurz-Str. 19a" \
--from-literal=ort="Paderborn"
```

- *ConfigMap* aus einer YAML-Datei erstellen:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: firma-config
data:
  firma: ORDIX AG
  strasse: Karl-Schurz-Str. 19a
  ort: Paderborn
```

```
Name:          firma-config
Namespace:     user-sk
Labels:        <none>
Annotations:   <none>
```

```
Data
====
firma:
----
ORDIX AG

ort:
----
Paderborn

strasse:
----
Karl-Schurz-Str. 19a
```

- *configMapRef* stellt alle Key/Value-Paare aus einer *ConfigMap* als **Umgebungsvariable** bereit.
- Wird in der Konfiguration des Containers angegeben:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmapref-test-pod
spec:
  containers:
    - name: configmapref-test-container
      image: busybox:1.31.0
      command: [ "/bin/sh", "-c", "env | grep 'firma'" ]
      envFrom:
      - configMapRef:
        name: firma-config
      restartPolicy: Never
```

- *configMapKeyRef* ermöglicht, den Namen der Umgebungs-Variablen selbst zu bestimmen.
- Wird in der Konfiguration des Containers angegeben:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmapkeyref-test-pod
spec:
  containers:
    - name: configmapkeyref-test-container
      image: busybox:1.31.0
      command: [ "/bin/sh", "-c", "env | grep 'FIRMA_NAME'" ]
      env:
        - name: FIRMA_NAME
          valueFrom:
            configMapKeyRef:
              name: firma-config
              key: firma
      restartPolicy: Never
```

ConfigMaps – Nutzung – Datei im Volume

- *ConfigMaps* können als Dateien im **Volume** bereitgestellt werden.
- Der Key wird zum Dateinamen.
Der Value zum Inhalt der Datei.

Hinweis

- Mehrzeilige Angaben in yaml mit '|'.
- *kubectl logs* verschluckt manchmal die letzte Zeile!

```
apiVersion: v1
kind: Pod
metadata:
  name: use-config-map-mount
spec:
  containers:
    - name: firma-container
      image: busybox:1.31.0
      command:
        - /bin/sh
        - -c
        - |
          ls -l /config
          cat /config/firma
          echo "=== done ==="
      volumeMounts:
        - name: firma-config-volume
          mountPath: /config
  volumes:
    - name: firma-config-volume
      configMap:
        name: firma-config
      restartPolicy: Never
```

- *volumeMounts*: definiert Mount-Prozess
 - *name*: Namen des Volumes
 - *mountPath*: Mount-Point
 - *subPath*: Name des keys der ConfigMap
-
- *subPath* ist erforderlich, wenn einzelne Datei gemountet werden soll.
Dann muss *mountPath* den Dateinamen angeben.

```
...  
  volumeMounts:  
    - name: config-volume  
      mountPath: /app/config/application.properties  
      subPath: application.properties  
...  
  volumes:  
    - name: config-volume  
      configMap:  
        name: app-config
```

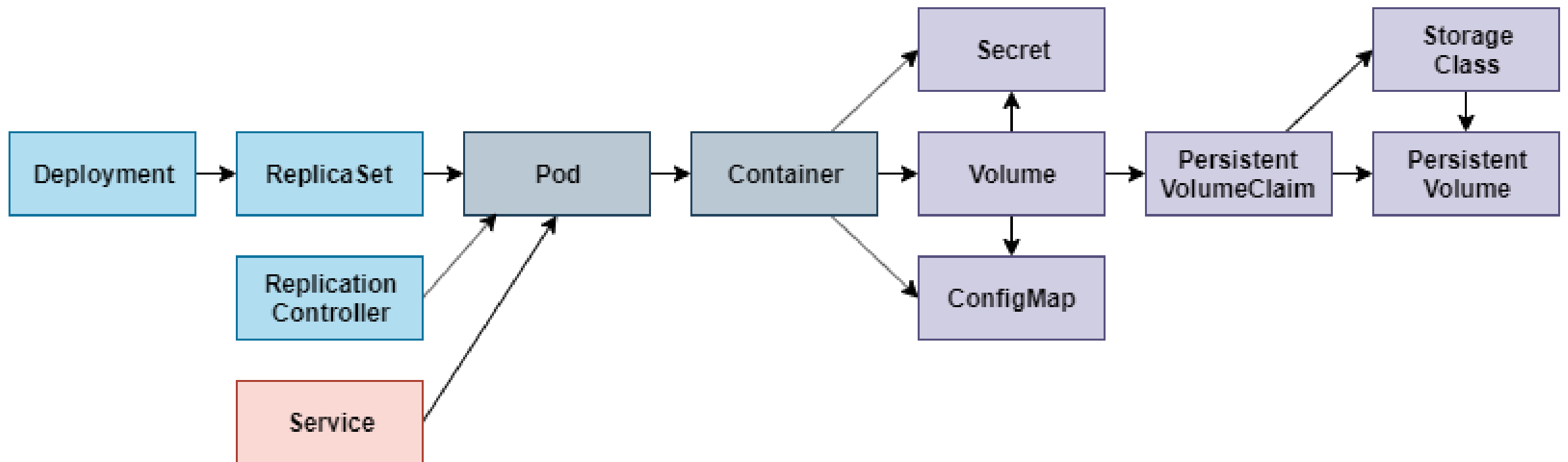
ConfigMaps – Nutzung – Mehrere Dateien mounten

- Die Datei index.html wird durch die Config-Map nginx-config ersetzt.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  index.html: |
    <html>
      <body>
        <h1>Hallo aus der ConfigMap</h1>
      </body>
    </html>
  name: nginx-config
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-demo
spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
      volumeMounts:
        - name: html-volume
          mountPath: /usr/share/nginx/html/index.html
          subPath: index.html
  volumes:
    - name: html-volume
      configMap:
        name: nginx-config
```

Secrets – Zusammenhang



- *Secrets* ermöglichen es ,**schützenswerte Informationen** (Passwörter,Privat Keys, ...) zu **speichern**.
- **Erstellung und Nutzung** von *Secrets* ähnelt der von *ConfigMaps*.
- Um ein *Secret* zu nutzen, muss es vom Pod referenziert werden.
- Dies kann auf 2 Arten erfolgen:
 - Environment-Variable
 - Datei in einem *Volume*
- **Achtung:**
 - Die **Secret-Informationen werden standardmäßig im Klartext im etcd gespeichert**.
 - Der kube-api-server verschlüsselt die Informationen, wenn er mit der Option `--encryption-provider-config`, die auf eine EncryptionConfiguration verweist, gestartet wird.

- Verwandtschaft von *Secrets* und *ConfigMaps* wird bei der Erstellung deutlich:

```
# kubectl create secret generic test-secret --from-literal=pwd=ordix123
# kubectl create secret generic test-secret --from-file=<dir|file>
# kubectl create secret generic test-secret --from-env-file=<file>
```

- *Secrets* werden standardmäßig **Base64** kodiert abgelegt:

```
# kubectl get secret test-secret -o yaml
apiVersion: v1
data:
  pwd: b3JkaXgxMjM=
kind: Secret
...

# echo "b3JkaXgxMjM=" | base64 -d
ordix123
```

- *Secret* als **Environment-Variable** bereitstellen:

```
...
  env:
  - name: SECRET_USERNAME
    valueFrom:
      secretKeyRef:
        name: mysecret
        key: username
...
```

- *Secret* als **Volume** einbinden:

```
...
  volumes:
  - name: secret-volume
    secret:
      secretName: test-secret
      defaultMode: 0400
...
```

Secret – Beispiel – Secrets aus Environment-Datei

- Secrets können auch aus Dateien gelesen werden. So lassen sie sich versioniert ablegen.
- *--from-env-file*: kubectl interpretiert Datei als Properties-Datei.
- Weder *describe* noch *get -o yaml* geben die Secrets im Klartext aus.
- *-o jsonpath*: Gibt das Attribut aus dem Manifest aus.

```
$ cat secret.properties
```

```
user=admin  
pwd=ordix123
```

```
$ kubectl create secret generic test-secret \  
--from-env-file=secret.properties  
secret/test-secret created
```

```
$ kubectl describe secret test-secret
```

```
...
```

```
Data
```

```
====
```

```
pwd:      8 bytes
```

```
user:     5 bytes
```

```
$ kubectl get secret test-secret \  
-o jsonpath="{.data.pwd}" | base64 -d  
ordix123sk
```

Secret – Beispiel – Manifest manuell erstellen

- Manifest für secrets können auch manuell erstellt werden.
- Dazu werden die Daten zunächst manuell encoded.
- Durch '-n', wird ein '\n' am Ende verhindert.

```
$ cat create-base46-encoding.sh
echo -n $1 | base64

$ ./create-base46-encoding.sh admin
YWRtaW4=
$ ./create-base46-encoding.sh ordix123
b3JkaXgxMjM=

$ cat test2-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: test2-secret
data:
  pwd: b3JkaXgxMjM=
  user: YWRtaW4=
```

Secret – Beispiel – Secret-Datei mounten

- Manche Anwendungen lesen ihre Secrets aus Dateien.
- Diese Dateien müssen bestimmte Zugriffsrechte haben.

```
$ cat secret.properties | base64
dXNlcj1hZG1pbGp2Q9b3JkaXgxMjMKCg==

$ cat file-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-file
data:
  secret.properties: |
    dXNlcj1hZG1pbGp2Q9b3JkaXgxMjMKCg==
```

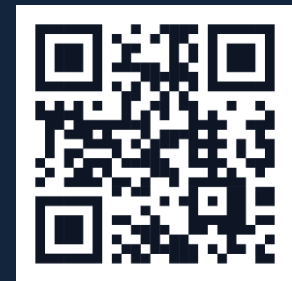
```
apiVersion: v1
kind: Pod
metadata:
  name: secretvolume-test-pod
spec:
  containers:
    - name: secretvolume-test-container
      image: busybox
      command: [ "/bin/sh", "-c", "ls -lL /secret/ &&
cat /secret/secret.properties ; echo '\nready'" ]
      volumeMounts:
        - name: test-secret-volume
          mountPath: /secret
  volumes:
    - name: test-secret-volume
      secret:
        secretName: secret-file
        defaultMode: 0400
  restartPolicy: Never
```

Secret – Beispiel – Secret-Erstellung im Klartext

- Secrets lassen sich mit dem Attribut *stringData* auch im Klartext erstellen!
- Dann tauchen die Kennwörter im Manifest im Klartext auf!
Werden wahrscheinlich im git im Klartext hinterlegt.
- Oftmals verstößt das gegen die Policy, dass Kennwörter nicht im Klartext verwendet werden sollen.

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-file
stringData:
  secret.properties: |
    user=admin
    pwd=ordix123
```

**Vielen Dank für
Ihre Aufmerksamkeit**



ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00
Fax: 0180 1 67349 0

seminare@ordix.de
<https://www.seminare.ordix.de/>